

SOFTWARE REQUIREMENTS SPECIFICATION

BUILDWISE*Modern Construction Management & Leftover Material Marketplace**" Build Smarter. Waste Less. "*

Document Type	Software Requirements Specification (SRS)
Project	Buildwise — Construction Management & Leftover Material Marketplace
SDG Alignment	SDG 12 — Responsible Consumption & Production (Target 12.5) SDG 9 — Industry, Innovation & Infrastructure (Target 9.4)
Version	1.0
Status	Draft for Review

Table of Contents

Section	Title	Page
1.	Introduction (Purpose, Scope, Audience, Acronyms, References)	3
2.	Overall Description (Perspective, Features, Roles, Constraints, SDGs)	5
3.	System Architecture (Layers & Fig. 3.1 Architecture Diagram)	7
4.	Functional Requirements (FR-01 to FR-21 Priority Table & Specs)	8
5.	Use Cases (Actors, Fig. 5.1 Use Case Diagram & Detailed Specs)	9
6.	System Workflow (Fig. 6.1 Flowchart & Fig. 6.2 Sequence Diagram)	11
7.	Data Structure & Logic Design (Context, Formulas, Fig. 7.1 Flow)	13
8.	Data Storage Design (LocalStorage Entities, Fig. 8.1 Schema)	14
9.	Non-Functional Requirements (Performance, Reliability, Usability)	15
10.	External Interface Requirements (UI, Hardware, Software, Comm)	15
11.	System Constraints & Assumptions	16
12.	Risk Analysis & Mitigation	16
13.	Future Enhancements (Planned Cloud, AI, and Payments)	17
14.	Appendix (Glossary & Technology Stack)	17
RTM	Requirements Traceability Matrix	18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Buildwise**, an integrated modern construction management platform and circular leftover material marketplace. Buildwise addresses structural construction site inefficiencies by consolidating daily project oversight, real-time material inventory tracking, and workforce attendance into a single responsive web system, while introducing a circular economy marketplace to turn surplus construction leftovers into recoverable capital. This document guides system design, implementation, and academic evaluation, serving as the formal technical specification of the current prototype.

1.2 Scope

Buildwise is implemented as a client-side Single Page Application (SPA). The current system scope covers:

- **Construction Site Management:** Multi-site progress monitoring, milestone completion tracking, budget vs. expenditure auditing, and site health status indicators.
- **Material Tracking & Usage:** Categorized inventory tracking (Cement, Steel, Tiles, Plumbing, Electrical, Aggregates), recording daily site consumption, dynamic remaining stock calculation, and automated low-stock warnings.
- **Workforce Management:** Crew member directory categorized by trade skill, daily wage rates, contact details, and historical attendance rates.
- **Digital Attendance:** 1-Click Mobile Punch Card check-in/out and simulated entrance QR code scanning with live muster roll compilation.
- **Task Management:** Operational task creation, priority assignment, tradesman allocation, and Kanban workflow status tracking.
- **Leftover Material Marketplace:** Direct conversion of logged site surplus into live B2B marketplace listings, city/category filtering, and buyer purchase inquiries.
- **Reporting Engine:** Vector PDF document generation for official muster rolls, material efficiency audits, and multi-site summaries, alongside CSV data exports.
- **Activity & Notification Stream:** Live chronological feed of operational site events and high-priority stock warnings.

Out of scope for the current prototype: Centralized server backend, remote relational database, user authentication (JWT/OAuth), live camera stream hardware access, and online payment gateways. These items are strictly categorized under Section 13: Future Enhancements.

1.3 Intended Audience

- **Project Developers:** For understanding component hierarchy, state contracts, and business logic.
- **Academic Evaluators & Mentors:** For reviewing software engineering rigor, architectural decisions, and SDG alignment.
- **Future Contributors:** For extending the current frontend prototype with cloud backend APIs and database integrations.

1.4 Definitions, Acronyms & Abbreviations

Term	Meaning
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
SPA	Single Page Application
API	Application Programming Interface
DOM	Document Object Model
PDF	Portable Document Format
CSV	Comma-Separated Values
QR Code	Quick Response Matrix Barcode
RBAC	Role-Based Access Control
SDG	Sustainable Development Goal (United Nations 2030 Agenda)
KPI	Key Performance Indicator
PPC	Portland Pozzolana Cement
TMT	Thermo-Mechanically Treated (reinforcement steel bars)
CPVC	Chlorinated Polyvinyl Chloride (plumbing pipe material)
Muster Roll	Formal register documenting worker attendance and daily wages
Brass	Traditional South Asian unit of aggregate volume (100 cubic feet)

1.5 References

- **React 19 Documentation** — react.dev
- **Vite 8 Build Tool** — vite.dev
- **jsPDF & AutoTable Documentation** — raw.githubusercontent.com/MrRio/jsPDF/master/docs/
- **United Nations SDG 12** — Responsible Consumption and Production (Target 12.5)
- **United Nations SDG 9** — Industry, Innovation and Infrastructure (Target 9.4)

1.6 Document Overview

Section 2 describes the product at a high level; Section 3 presents the system architecture; Section 4 lists functional requirements; Section 5 provides use case specifications and diagram; Section 6 outlines operational system workflows and sequence diagrams; Section 7 covers data structures and application logic; Section 8 defines client-side data storage; Sections 9–12 establish non-functional requirements, external interfaces, constraints, and risk analysis; Sections 13–14 detail future scope and appendix material; concluding with the Requirements Traceability Matrix.

2. Overall Description

2.1 Product Perspective

In standard construction practice, 10–15% of procured structural materials are discarded as jobsite waste, while site managers juggle fragmented paper registers for attendance and procurement. Buildwise provides a self-contained, domain-specific operating system that unites jobsite management with a circular economy mechanism. Buildwise connects operational consumption tracking directly with an integrated B2B leftover marketplace, allowing contractors to monetize surplus materials rather than hauling them to landfills.

2.2 Product Features (at a glance)

- **Executive Dashboard:** Live KPI stat cards (Active Sites, Crew Present, Stock Volume, Pending Tasks), usage trend graphs, and attendance distribution donut charts.
- **Multi-Site Management:** Real-time tracking of site progress percentages, supervisors, budgets, expenditures, and risk alerts.
- **Material Inventory Tracking:** Comprehensive tracking by trade categories with daily usage logging, safety thresholds, and low-stock indicators.
- **Waste → Value Leftover Monetizer:** 1-Click tool converting identified surplus items directly into active marketplace listings.
- **Leftover Resale Marketplace:** Searchable B2B trading hub with category filters, city filters (Nashik, Pune, Mumbai, Bangalore), condition badges, and buyer inquiry dispatches.
- **Workforce Directory:** Specialized trade skill tracking (Masonry, Steel, Carpentry, Electrical, Plumbing, Labor), daily wages, and contact cards.
- **Dual-Mode Attendance:** 1-Click Mobile Punch Card check-in/out and an interactive simulated entrance QR Code Scanner.
- **Operational Task Kanban:** 4-stage task progression board (To Do, In Progress, Review, Completed) with worker assignments.
- **Automated Document Generation:** Client-side vector PDF compilation for muster rolls, material efficiency audits, and site summaries, with CSV export.
- **Role Perspective Switcher:** Instant top-bar persona switching between Builder, Site Engineer, and Worker modes.

2.3 User Classes and Characteristics

User Class	Characteristics / Main Functions
Builder / Director	Executive persona. Monitors multi-site milestones, financial expenditures, aggregate inventory yields, and recovers capital via the leftover marketplace.
Site Engineer	Operational supervisor. Records daily material consumption quantities, inspects ongoing structural tasks, verifies crew muster logs, and flags leftovers for resale.
Site Worker	Frontline tradesman. Uses a distraction-free mobile portal for 1-click attendance punch, entrance QR scanning, and viewing assigned tasks.

2.4 Operating Environment

- **Client Application:** Modern Chromium- or Firefox-based web browser with ECMAScript 2022+ support.
- **Runtime & Build Platform:** React 19 Single Page Application built with Vite 8.2.0.
- **Persistence Layer:** Browser LocalStorage API (zero remote server requirement in current prototype).
- **Styling Environment:** Pure Vanilla CSS design tokens and layout classes without utility framework dependencies.

2.5 Design and Implementation Constraints

- **Client-Side Persistence:** Application data resides entirely in browser `localStorage` and does not sync across separate physical devices.
- **Simulated Hardware Interface:** QR code scanning uses an animated viewfinder simulation rather than native camera device access.
- **Simulated Authentication:** Role switching is managed via client-side Context state without encrypted credentials or tokens.
- **Browser Origin Storage Quota:** Total JSON persistence is bounded by the browser origin quota of 5 MB.

2.6 Assumptions and Dependencies

- The user operates a browser with HTML5 Canvas and Web Storage enabled.
- The browser supports dynamic Blob and Object URL generation for document downloads.
- External photographic assets depend on CDN availability (images.unsplash.com).

2.7 Alignment with United Nations SDGs

SDG 12: Responsible Consumption and Production (Target 12.5)

Substantially reduce waste generation through prevention, reduction, recycling, and reuse by converting leftover construction materials into marketplace listings.

SDG 9: Industry, Innovation and Infrastructure (Target 9.4)

Upgrade infrastructure and modernize construction workflows through client-side digitization and material utilization optimization.

3. System Architecture

3.1 Layers

- **Presentation Layer:** React 19 functional components, view routers, and reusable atomic UI elements (Buttons, Modals, ProgressBars, StatCards).
- **Application State & Logic Layer (AppContext):** Centralized state provider managing reactive data entities, threshold calculations, and toast dispatches.
- **Client-Side Services:** Vector PDF generation engine (`pdfGenerator.js`), CSV tabular serializer (`exportCSV`), and celebration animations (`canvas-confetti`).
- **Persistence Layer:** Browser LocalStorage maintaining 8 discrete JSON storage collections.
- **Future Cloud Layer (Planned):** Cloud REST/WebSocket backend, PostgreSQL database, authentication, and AI inference microservices.

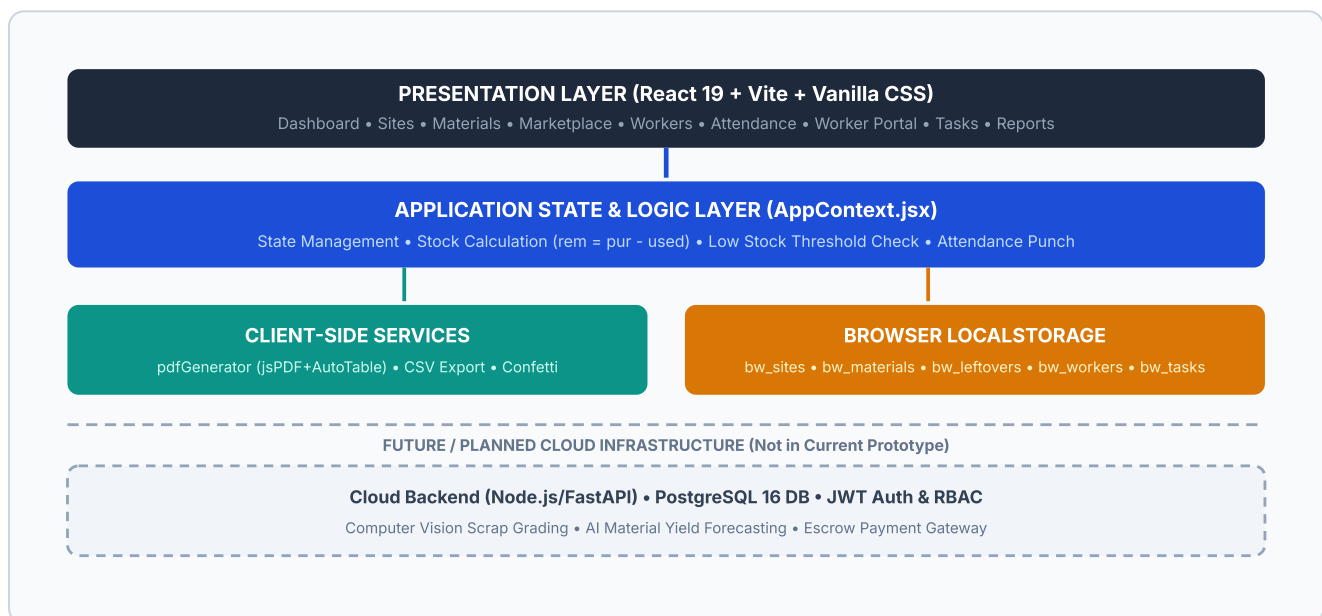


Fig. 3.1 — Layered architecture: Presentation → AppContext → Services → LocalStorage / Future Cloud

4. Functional Requirements

4.1 Requirement List

ID	Requirement Description	Priority
FR-01	System shall allow instant role perspective switching between Builder, Engineer, and Worker modes.	High
FR-02	System shall display real-time dashboard KPI metrics for active sites, crew attendance, inventory, and tasks.	High
FR-03	System shall manage multi-site projects with milestone progress bars, budgets, expenditures, and health badges.	High
FR-04	System shall allow adding new construction projects with supervisor allocations and budget metadata.	Medium
FR-05	System shall track categorized material stock with purchased, used, and dynamically calculated remaining quantities.	High
FR-06	System shall record daily material consumption, deducting used units and blocking entries exceeding remaining stock.	High
FR-07	System shall automatically trigger a "Low Stock" status when remaining inventory falls at or below threshold.	High
FR-08	System shall provide a 1-click action to list surplus leftover materials directly onto the B2B marketplace.	High
FR-09	System shall display surplus material listings with category, price per unit, total price, condition, and location.	High
FR-10	System shall allow posting new marketplace listings with custom category, condition, unit pricing, and photo assets.	Medium
FR-11	System shall provide a buyer inquiry dialog to send simulated purchase requests to surplus material sellers.	Medium
FR-12	System shall manage a trade crew directory with skill designations, daily wage rates, and historical attendance rates.	High
FR-13	System shall provide a 1-Click Mobile Punch Card enabling workers to check in and check out with timestamping.	High
FR-14	System shall simulate entrance QR code scanning with an animated laser viewfinder to verify worker attendance.	High
FR-15	System shall maintain a daily attendance muster register logging worker name, trade, site, timestamp, and method.	High
FR-16	System shall render a dedicated mobile worker portal for simplified attendance punching and personal task review.	High
FR-17	System shall provide a 4-stage Kanban board (To Do, In Progress, Review, Completed) for daily task operations.	High

ID	Requirement Description	Priority
FR-18	System shall generate and download vector PDF reports for Muster Rolls, Material Efficiency Audits, and Sites Summaries.	High
FR-19	System shall export tabular muster roll and inventory records into standardized CSV spreadsheet files.	High
FR-20	System shall provide a chronological notification stream and high-priority stock warning cards.	Medium
FR-21	System shall allow updating user profile preferences, regional measurement units (Brass vs. Tons), and demo data reset.	Medium

5. Use Cases

5.1 Actors

The system defines three primary actors: **The Builder** (executive portfolio oversight and capital recovery), **The Site Engineer** (field material usage, task assignment, and crew supervision), and **The Site Worker** (frontline attendance check-in and task execution).

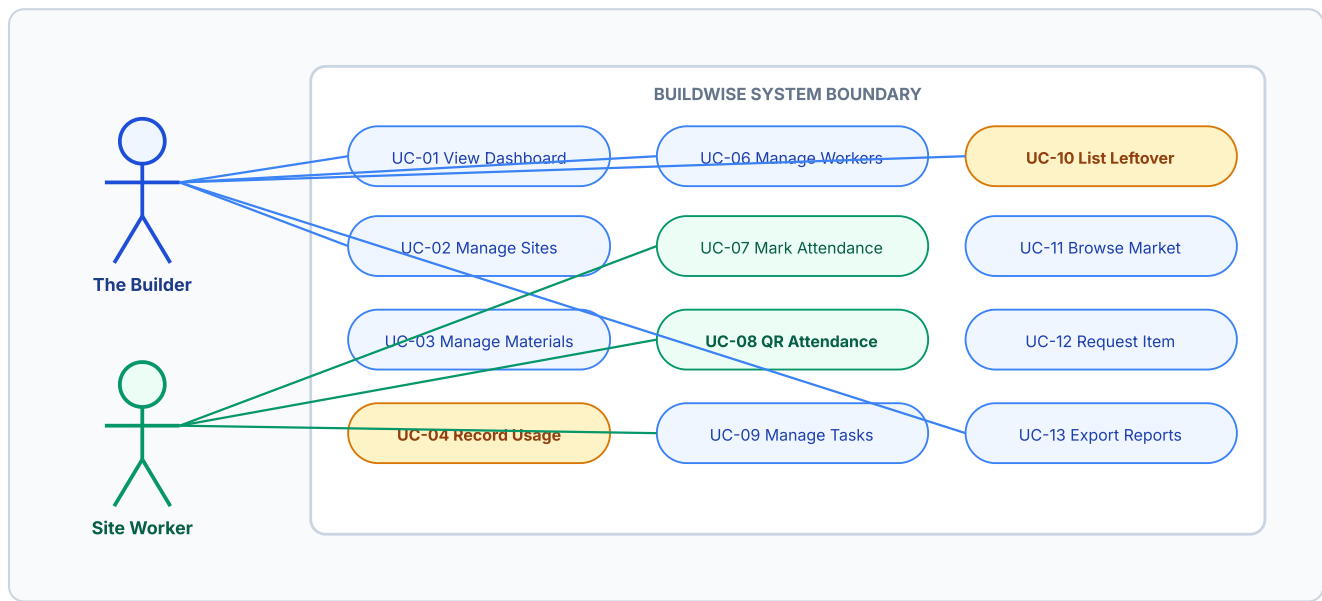


Fig. 5.1 — Primary actors (Builder, Site Engineer, Worker) and their system interactions

5.2 Detailed Use Case Specifications

UC-04: Record Material Usage

- **Actor:** Site Engineer
- **Purpose:** Log consumed materials on-site, dynamically update stock balance, and trigger reorder alerts if stock drops below threshold.
- **Preconditions:** Material exists in `bw_materials` with remaining stock > 0 .
- **Main Flow:**
 1. Engineer opens the Materials view or clicks "+ Record Usage" on Dashboard.
 2. Engineer selects material, views current remaining balance and site location.
 3. Engineer enters consumed quantity and optional activity notes.
 4. Engineer clicks "Confirm Usage".
 5. System updates used and remaining quantities: $remaining = purchased - used$.
 6. System checks if $remaining \leq threshold$; marks status as "Low Stock" if triggered.
 7. System saves updated state to `LocalStorage` (`bw_materials`).
 8. System appends an entry to `bw_activities` and displays a success toast.
- **Alternative Flow:**
 - 3a. Engineer inputs quantity exceeding available stock.
 - 3b. System shows error "*Cannot record more than available remaining stock!*" and blocks submission.
- **Postconditions:** Inventory balance updated; low-stock alerts rendered if threshold is reached.

UC-08: Verify Attendance (QR Scanner)

- **Actor:** Site Worker / Site Engineer
- **Purpose:** Register worker presence at a construction site entrance via simulated QR code scan.
- **Preconditions:** Worker profile exists in `bw_workers`.
- **Main Flow:**
 1. User navigates to Attendance view or Worker Portal and clicks "Scan Site QR".
 2. System displays `QRScannerModal` with animated camera viewfinder.
 3. User selects site and clicks "Simulate Camera Scan".
 4. System executes 1400ms scanning laser animation.
 5. System marks worker `isCheckedIn = true` and records punch timestamp.
 6. System appends record to `bw_attendance` with method: "Site QR Scan".
 7. Confetti triggers, confirmation message displays, and `LocalStorage` updates.
- **Postconditions:** Worker registered as Present in the official muster roll.

UC-10: List Leftover Material (Waste → Value)

- **Actor:** Builder / Site Engineer
- **Purpose:** Transform site surplus into active marketplace listings to recover capital.

- **Preconditions:** Item exists in `bw_leftovers`.
- **Main Flow:**
 1. User navigates to "Materials You Can Recover Value From" section.
 2. User reviews surplus items, conditions, and estimated monetary values.
 3. User clicks "Sell on Marketplace" on selected item.
 4. System removes item from `bw_leftovers` and generates a listing in `bw_marketplace`.
 5. Confetti triggers; system posts activity log and displays success toast with recovered value.
- **Postconditions:** Surplus item is listed for sale in the B2B marketplace.

6. System Workflow

6.1 Material Management Workflow

Material → Record Usage → Update Quantity → Calculate Remaining Stock → Check Threshold → Low Stock Alert

6.2 Attendance Workflow

Worker → Punch / QR Verification → Attendance Record → Attendance Status (Present)

6.3 Task Workflow

Task → To Do → In Progress → Review → Completed

6.4 Leftover Marketplace Workflow

Leftover Material → Create Listing → Marketplace → Buyer Request

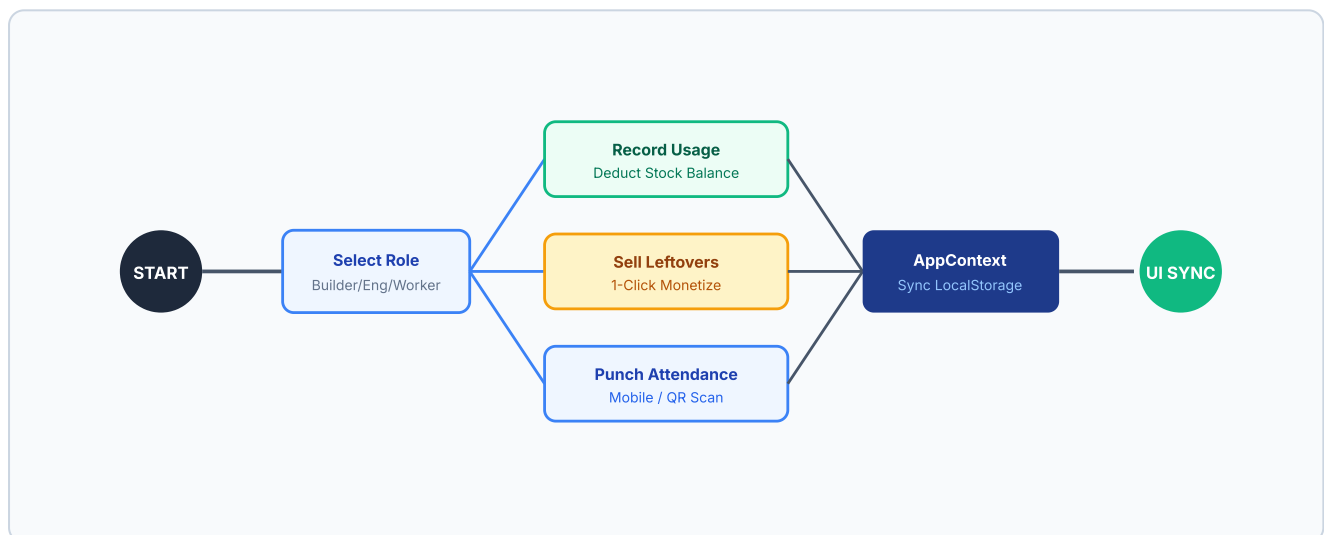


Fig. 6.1 — Step-by-step Buildwise operational flow from role intake to LocalStorage persistence

6.2 Sequence Diagram: Record Material Usage

The sequence diagram below traces the representative operational workflow of logging daily material consumption through the React interface, AppContext computation, LocalStorage persistence, and toast feedback.

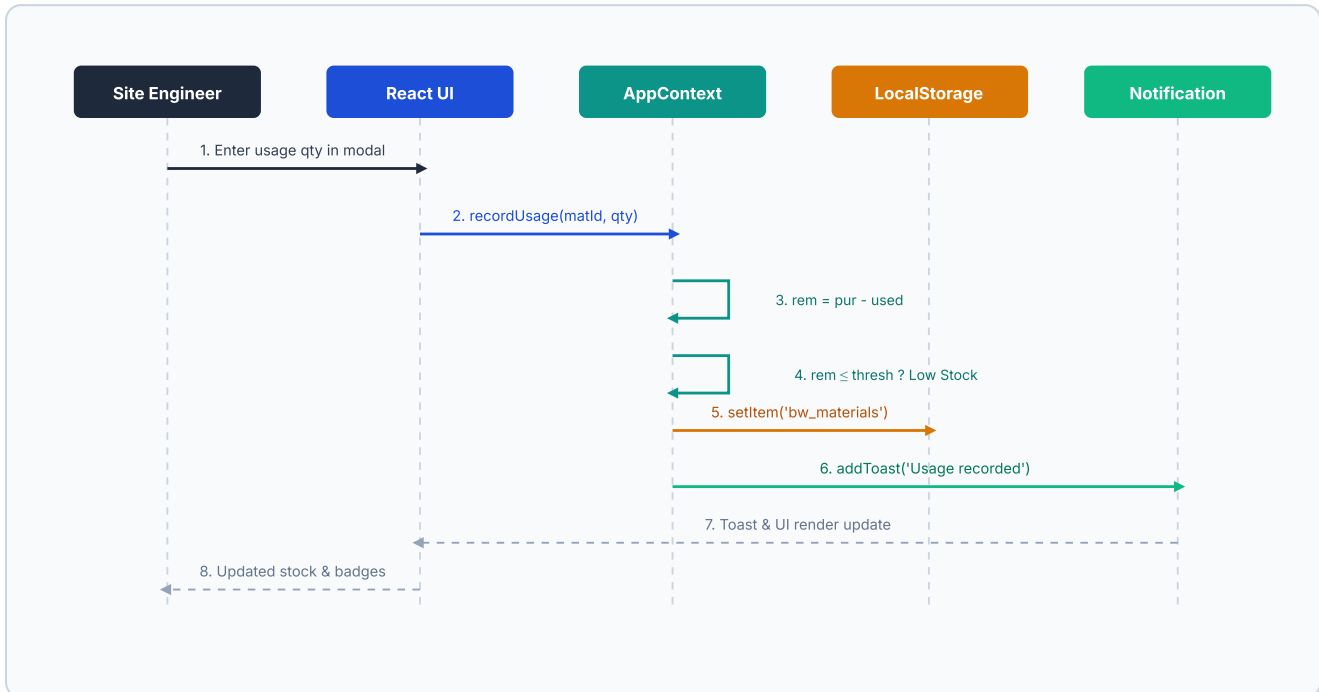


Fig. 6.2 — End-to-end message flow for a material consumption logging event

7. Data Structure & Application Logic Design

7.1 Centralized Application State via React Context

The application state is maintained in `AppContext.jsx` using React 19 Context. On initialization, each state variable performs a lazy read from `localStorage`, falling back to seed mock collections:

```
const [materials, setMaterials] = useState(() => {
  const saved = localStorage.getItem('bw_materials');
  return saved ? JSON.parse(saved) : initialMaterials;
});
```

Eight `useEffect` hooks listen to changes on their respective state vectors and serialize updates back to `localStorage`.

7.2 Mathematical & Computational Logic

- **Material Balance Equation:** $remaining = purchased - used$
- **Threshold Stock Status:** $status = remaining \leq threshold ? "Low Stock" : "In Stock"$
- **Attendance Rate:** $Rate (\%) = (Present\ Workers / Total\ Enrolled\ Workers) \times 100$
- **Resale Value Recovery:** $Value = \sum estimatedValue\ of\ monetized\ leftovers$

7.3 Algorithmic Data Structures

- **Array Collections:** Sequential collections for materials, workers, tasks, sites, and marketplace listings, enabling $O(n)$ filtering and mapping.
- **Lookup Map:** Instant role profile resolution (`currentUser[userRole]`) in $O(1)$ constant time.
- **FIFO Buffer:** Recent activity stream capped at 16 entries (`[newAct, ...prev.slice(0, 15)]`).
- **Expiring Queue:** Toast notifications auto-dismissing after a 4000ms timer.

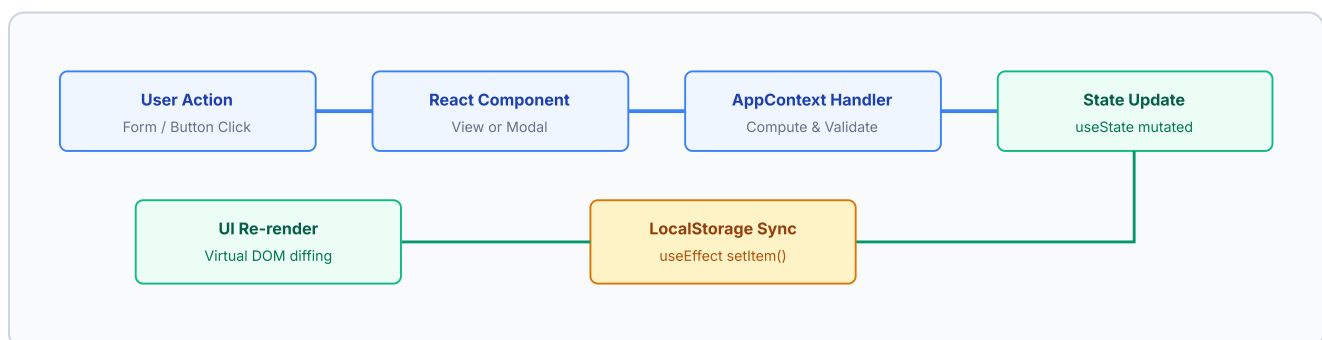


Fig. 7.1 — Each core data structure and state transition mapped to application logic

8. Data Storage Design

8.1 Core Entities

Entity	Storage Key	Purpose	Important Fields
Sites	bw_sites	Active construction projects	id, name, location, progress, budget, spent, health
Materials	bw_materials	Material inventory stock	id, name, category, purchased, used, remaining, status, threshold
Leftovers	bw_leftovers	Identified jobsite surplus	id, title, quantity, estimatedValue, condition, site
Marketplace	bw_marketplace	B2B surplus listings	id, title, price, totalPrice, city, seller, sellerRating
Workers	bw_workers	Trade crew directory	id, name, trade, wagePerDay, isCheckedIn, checkInTime
Tasks	bw_tasks	Operational site tasks	id, title, workerName, priority, progress, status
Attendance	bw_attendance	Shift muster roll	id, workerName, siteName, time, date, status, method
Activities	bw_activities	Chronological audit logs	id, text, time, type, statusClass

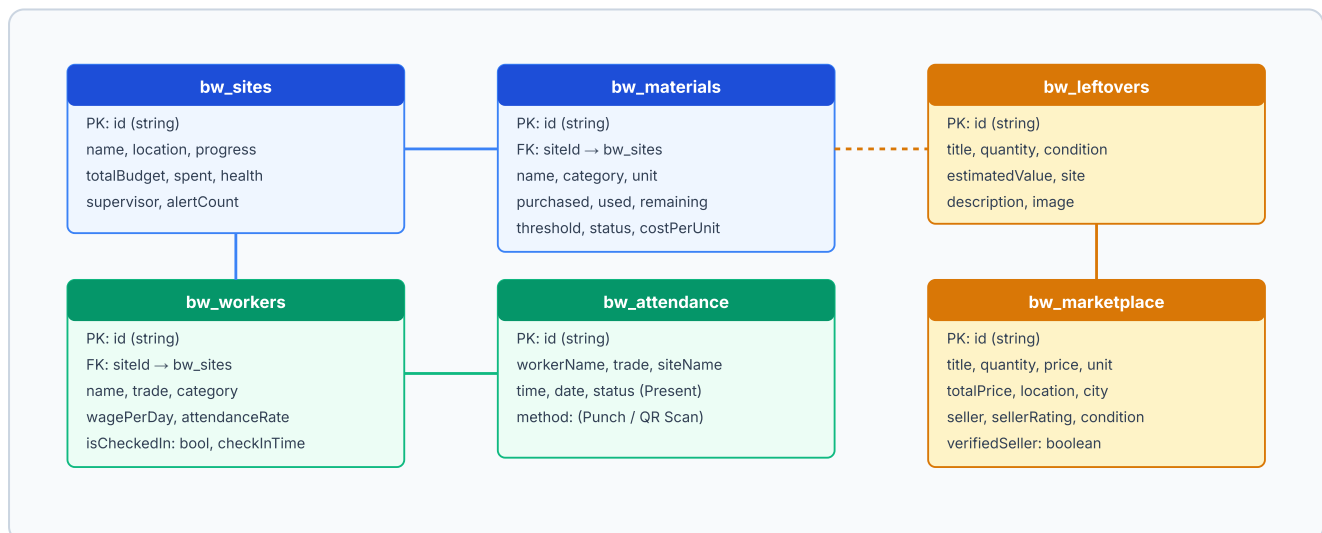


Fig. 8.1 — Client-side LocalStorage entity relationships and schema dictionary

9. Non-Functional Requirements

Category	Requirement Specification
Performance	Initial SPA bundle load ≤ 1.5 s on broadband; state recalculations execute in ≤ 16 ms (60 fps).
Usability	Interface adheres to high-contrast construction styling (Navy, Amber, Emerald); touch targets ≥ 48 px.
Reliability	Atomic state sync to <code>localStorage</code> prevents data loss on accidental browser refreshes.
Maintainability	Codebase maintains zero syntax errors and passes static code analysis via Oxcint.
Responsiveness	Adaptive layout for desktop (>1024 px), tablet (768–1024px), and mobile devices (<768 px).
Compatibility	Consistent rendering across evergreen browsers (Chrome, Firefox, Safari, Edge).
Data Persistence	Total JSON persistence footprint < 250 KB, within the 5MB origin storage limit.
Security	Input sanitization via React JSX escaping prevents Cross-Site Scripting (XSS).

10. External Interface Requirements

10.1 User Interface

The UI features a dual-column layout with a 260px collapsible sidebar, breadcrumb top navigation with role toggles, modal dialogs for all data entry, and interactive visual cards.

10.2 Hardware Interfaces

Standard mouse, keyboard, and touch display. *Camera hardware streaming is simulated in the current prototype via animated SVG viewfinders.*

10.3 Software Interfaces

React 19.2.8, Vite 8.2.0, jsPDF 4.2.1, jspdf-autotable 5.0.8, lucide-react 1.33.0, canvas-confetti 1.9.4, and Oxcint 1.75.0.

10.4 Communication Interfaces

The current prototype operates purely client-side; no remote server communication layer or WebSocket connections are initiated.

11. System Constraints & Assumptions

- **Browser-Based Application:** Operates entirely in the client browser without a centralized backend server.
- **LocalStorage Persistence:** State is tied to a single device/browser origin; clearing browsing cache resets data to mock seeds.
- **Simulated Security:** Role switching is a client-side state variable for demonstration purposes without password enforcement.
- **Verified Truth:** No PostgreSQL, MongoDB, or Supabase instance exists in the current implementation.

12. Risk Analysis & Mitigation

Risk	Impact	Mitigation
LocalStorage Data Purge	User clears browser cache, causing loss of newly added materials.	Seed data fallback in <code>mockData.js</code> ; "Reset Demo Data" restores initial state cleanly.
Negative Material Stock Overflow	Engineer enters consumption quantity greater than remaining stock.	Modal input validation bounds input: <code>usedAmount ≤ remaining</code> , disabling confirmation on overflow.
Single-Device Concurrency	Two engineers cannot share state across different physical devices.	Documented limitation; targeted for cloud PostgreSQL/WebSocket resolution in future roadmap.
Unverified QR Entrance Check-ins	Workers could simulate check-in without being physically present.	Future planned release will bind QR validation to HTML5 Geolocation geofencing.

13. Future Enhancements

IMPORTANT: The features listed below represent planned future scope and are NOT implemented in the current prototype.

- **Cloud Backend & Centralized Database:** Hosted FastAPI / Node.js backend with PostgreSQL 16 / Supabase storage.
- **Authentication & True RBAC:** Phone OTP verification, biometric sign-in, and JWT session authorization.
- **Multi-Device Real-Time Sync:** Bi-directional WebSocket channels keeping field workers and executive desktops in sync.
- **AI Material Prediction & Computer Vision:** Machine learning scrap volume estimation and CV scrap grading from mobile camera photos.
- **GPS Geofencing:** Automatic 50-meter perimeter verification for physical site check-ins.
- **Escrow Payment Gateway:** UPI and card payment escrow protecting contractor funds during marketplace material trading.

14. Appendix

14.1 Glossary

See Section 1.4 for the primary term glossary.

14.2 Technology Stack

Category	Technology	Purpose
Frontend	React 19.2.8	Declarative reactive UI framework
Build Tool	Vite 8.2.0	Development server and production bundler
Styling	Vanilla CSS	Design tokens, responsive layouts, no Tailwind
PDF Generation	jsPDF 4.2.1	Vector PDF rendering engine
PDF Tables	jspdf-autotable 5.0.8	Multi-column formatted PDF tables
Iconography	lucide-react 1.33.0	Clean SVG construction and utility icons
Animations	canvas-confetti 1.9.4	Gamified celebration feedback
Linter	Oxlint 1.75.0	High-speed Rust-based code linter
Storage	Web Storage API	Browser LocalStorage client-side persistence

Requirements Traceability Matrix

Req. ID	Feature	Module	Primary Source File
FR-01	Role Perspective Switcher	Navigation	src/components/layout/TopNav.jsx
FR-02	Executive KPI Stat Cards	Dashboard	src/components/dashboard/DashboardView.jsx
FR-03	Multi-Site Tracking	Sites	src/components/sites/SitesView.jsx
FR-04	Add Project Site	Sites	src/components/sites/AddSiteModal.jsx
FR-05	Material Stock Directory	Materials	src/components/materials/MaterialsView.jsx
FR-06	Material Usage Logging	Materials	src/components/materials/RecordUsageModal.jsx
FR-07	Low-Stock Alerting	Materials	src/context/AppContext.jsx
FR-08	Waste → Value Resale	Materials	src/components/materials/LeftoverSection.jsx
FR-09	Marketplace Feed	Marketplace	src/components/marketplace/MarketplaceView.jsx
FR-10	Post Surplus Listing	Marketplace	src/components/marketplace/PostListingModal.jsx
FR-11	Buyer Request Inquiry	Marketplace	src/components/marketplace/RequestListingModal.jsx
FR-12	Crew Workforce Directory	Workers	src/components/workers/WorkersView.jsx
FR-13	1-Click Punch Card	Attendance	src/components/attendance/AttendancePunchCard.jsx
FR-14	QR Code Attendance	Attendance	src/components/attendance/QRScannerModal.jsx
FR-15	Muster Register Log	Attendance	src/components/attendance/AttendanceView.jsx
FR-16	Worker Mobile Mode	Worker Portal	src/components/worker-portal/WorkerPortalView.jsx
FR-17	Operations Kanban	Tasks	src/components/tasks/TasksView.jsx
FR-18	PDF Muster Roll Export	Reports	src/utils/pdfGenerator.js
FR-19	CSV Data Export	Reports	src/utils/pdfGenerator.js
FR-20	Activity Log Stream	Notifications	src/components/notifications/NotificationsView.jsx
FR-21	Settings & Demo Reset	Settings	src/components/settings/SettingsView.jsx